

Laboratorio 6: Cliente SPA de Blueprints con React, Redux, Axios y JWT

Juan Miguel Rojas Chaparro, Cristian David Silva Perilla, David Eduardo Salamanca, Felipe Eduardo Calvache
Escuela Colombiana de Ingenieria Julio Garavito
Bogota, Colombia

Resumen—En este laboratorio se desarrollo un cliente SPA para Blueprints usando React + Vite, Redux Toolkit, Axios y autenticacion JWT. Se implementaron modulos de consulta y creacion de blueprints, render de puntos en canvas y conmutacion entre API real y servicio mock. El sistema se valido tecnicamente con pruebas automatizadas, analisis estatico y compilacion de produccion.

Index Terms—React, Redux Toolkit, Axios, JWT, Canvas, Vitest, Vite, SPA.

I. INTRODUCCION

El laboratorio moderniza el cliente tradicional de Blueprints hacia una arquitectura SPA mantenible. La implementacion separa interfaz, estado global y capa de servicios, permitiendo escalabilidad y pruebas automatizadas del flujo funcional.

II. OBJETIVOS

- Implementar una SPA para consultar, visualizar y crear blueprints.
- Gestionar estado global y asincronia con Redux Toolkit.
- Integrar autenticacion JWT en el cliente usando interceptores de Axios.
- Graficar blueprints en canvas a partir de sus puntos.
- Validar calidad tecnica con test, lint y build.

III. ARQUITECTURA DE LA SOLUCION

III-A. Capas principales

- **UI:** componentes de formulario, tabla/listado y canvas.
- **Paginas:** vista principal de blueprints, login y detalle.
- **Estado:** slice Redux con *thunks* asincronos.
- **Servicios:** API real (Axios) y mock en memoria con interfaz comun.

III-B. Conmutacion de origen de datos

```
const useMock = String(import.meta.env.VITE_USE MOCK || 'false').toLowerCase() === 'true'  
const blueprintsService = useMock ? apimock : blueprintsApiClient
```

Listing 1: Seleccion de servicio segun variable de entorno.

III-C. Integracion JWT en Axios

```
api.interceptors.request.use((config) => {  
  const token = localStorage.getItem('token')  
  if (token) {  
    config.headers.Authorization = token.startsWith('Bearer ')  
      ? token  
      : `Bearer ${token}`  
  }  
  return config  
})
```

Listing 2: Interceptor para enviar token Bearer.

IV. IMPLEMENTACION TECNICA

IV-A. Estado global

El estado principal incluye:

- **authors:** lista de autores.
- **byAuthor:** blueprints agrupados por autor.
- **current:** blueprint actualmente abierto.
- **Estados de flujo:** status, byAuthorStatus, currentStatus, createStatus.
- **error:** mensaje de error global para operaciones fallidas.

IV-B. Flujos asincronos

```
fetchAuthors  
fetchByAuthor  
fetchBlueprint  
createBlueprint
```

Listing 3: Thunks implementados en el slice.

IV-C. Visualizacion en canvas

El modulo de canvas:

- limpia y repinta el lienzo en cada cambio de puntos,
- dibuja cuadrícula de referencia,
- une puntos con segmentos consecutivos,
- marca vertices con circulos.

V. EVIDENCIAS DE FUNCIONAMIENTO

V-A. E1. Consulta de blueprints por autor

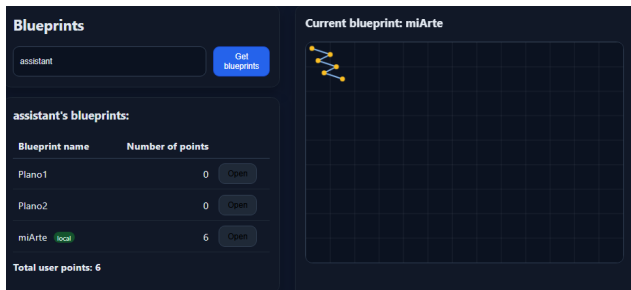


Figura 1: Consulta de blueprints por autor y despliegue de resultados en tabla.

V-B. E2. Apertura de blueprint y render en canvas



Figura 2: Apertura de blueprint seleccionado y render de puntos en canvas.

V-C. E3. Creacion de blueprint

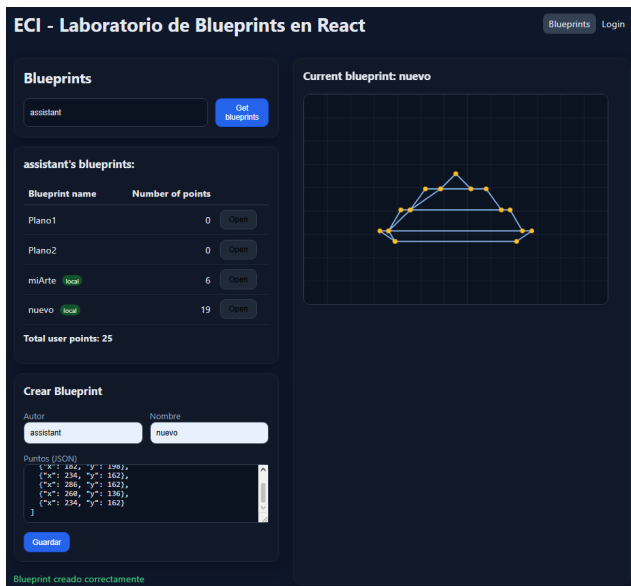


Figura 3: Formulario de creacion de blueprint con payload de puntos en JSON.

V-D. E4. Pruebas automatizadas

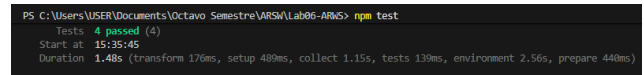


Figura 4: Ejecucion de Vitest con pruebas aprobadas.

V-E. E5. Lint y build de produccion

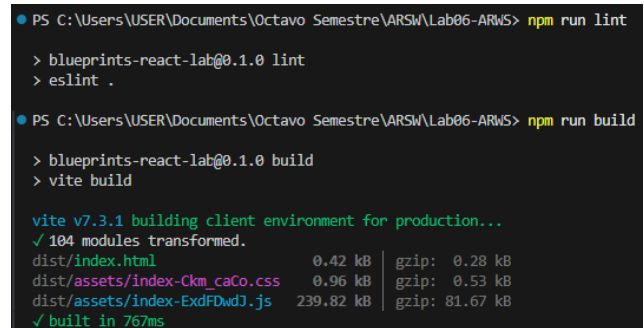


Figura 5: Validacion tecnica: ESLint y build de produccion exitosos.

V-F. E6. Login exitoso

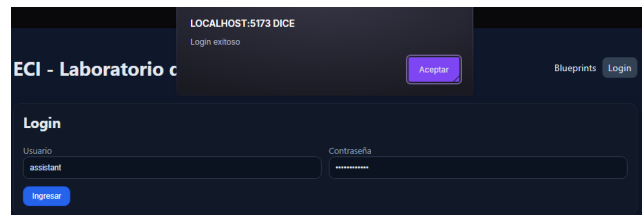


Figura 6: Autenticacion exitosa en la vista de login con respuesta positiva del sistema.

V-G. E7. Login fallido



Figura 7: Autenticacion fallida con manejo de error de credenciales invalidas en interfaz.

V-H. E8. Conmutacion de servicio mock

Se valido la capacidad de ejecutar la aplicacion con datos simulados sin dependencia del backend, cumpliendo el requerimiento de alternar entre servicio real y mock mediante variable de entorno.

Configuracion usada en entorno local:

```
VITE_USE MOCK=true
```

Listing 4: Activacion de mock por variable de entorno.

Procedimiento de validacion:

1. Configurar la variable de entorno con valor true.
2. Ejecutar el cliente en modo desarrollo.
3. Consultar un autor de prueba (por ejemplo: john).
4. Verificar listado de blueprints y apertura en canvas.
5. Confirmar que la aplicacion sigue operativa aun con backend apagado.

Resultado esperado y observado:

- Con mock activo, la consulta y visualizacion funcionan con datos en memoria.
- El flujo funcional principal no depende del API remoto.
- Se confirma que la interfaz de servicios es intercambiable sin cambios en la UI.

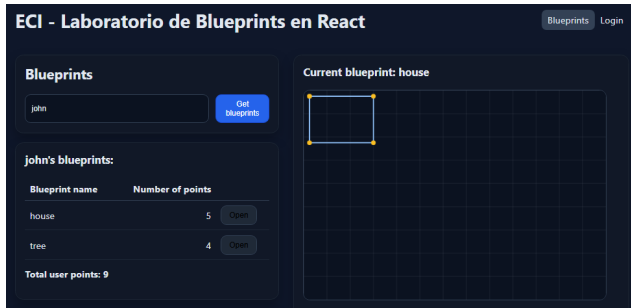


Figura 8: Aplicacion ejecutando consultas con $VITE_{USE MOCK} = true$.

VI. VALIDACION TECNICA

VI-A. Pruebas automatizadas

```
npm test
```

Listing 5: Comando de pruebas.

Resultado: 4 pruebas ejecutadas y 4 aprobadas.

VI-B. Analisis estatico

```
npm run lint
```

Listing 6: Comando de lint.

Resultado: ejecucion sin errores de ESLint.

VI-C. Compilacion

```
npm run build
```

Listing 7: Comando de build de produccion.

Resultado: compilacion completada y artefactos generados en dist.

VI-D. Matriz consolidada

Cuadro I: Matriz de validacion tecnica

ID	Validacion	Criterio de aceptacion	Estado
E1	Pruebas	Todas las pruebas en verde	Cumple
E2	Lint	Cero errores de ESLint	Cumple
E3	Build	Compilacion de produccion exitosa	Cumple

VII. CONCLUSIONES

1. Se implemento una SPA funcional para Blueprints con separacion clara por capas.
2. Redux Toolkit simplifico el manejo de estado asincrono y de errores.
3. La integracion de JWT via interceptores de Axios centralizo la autentificacion.
4. El canvas permitio visualizar correctamente la geometria de los blueprints.
5. La validacion con test, lint y build respalda la calidad tecnica de la entrega.

REFERENCIAS

- [1] React Documentation. [Online]. Available: <https://react.dev/>
- [2] Redux Toolkit Documentation. [Online]. Available: <https://redux-toolkit.js.org/>
- [3] Axios Documentation. [Online]. Available: <https://axios-http.com/>
- [4] Vite Documentation. [Online]. Available: <https://vite.dev/>
- [5] Vitest Documentation. [Online]. Available: <https://vitest.dev/>